



Protocol Audit Report

Version 1.0

Sid Hary

June 15, 2026

Protocol Audit Report

Sid Hary

June 15, 2026

Prepared by: Sid Hary Lead Security Researchers: - Sid Hary

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
 - High
 - * [H-1] Storing the Password On-Chain Makes It Publicly Accessible
 - * [H-2] Missing Access Control in `PasswordStore` : `setPassword` Allows Unauthorized Password Changes
 - Informational
 - * [I-1] Incorrect NatSpec Documentation for `getPassword`

Protocol Summary

PasswordStore is a protocol dedicated to storage and retrieval of a user's passwords. The protocol is designed to be used by a single user, and is not designed to be used by multiple users. Only the owner should be able to set and access this password.

Disclaimer

The Sid Hary team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

Scope

```
1 ./src/  
2 ----> PasswordStore.sol
```

Roles

- Owner: The user who can set the password and read the password.
- Outsiders: No one else should be able to set or read the password.

Executive Summary

Issues found

Severity	Number of Issues
High	2
Medium	0
Low	0
Info	1
Total	3

Findings

High

[H-1] Storing the Password On-Chain Makes It Publicly Accessible

Description:

All data stored on-chain is publicly accessible and can be read by anyone. The `PasswordStore::s_password` variable is intended to remain private and only be accessible to the owner through the `PasswordStore::getPassword` function.

Impact:

Any user can read the password directly from blockchain storage, completely compromising the protocol's intended privacy guarantees.

Proof of Concept (PoC):

The following steps demonstrate how any user can read the password directly from storage.

1. Start a local Anvil chain.

```
1 make anvil
```

2. Deploy the contract.

```
1 make deploy
```

3. Read storage slot 1.

We use slot 1 because `s_password` is stored at that location.

```
1 cast storage <ADDRESS_HERE> 1 --rpc-url http://127.0.0.1:8545
```

Example output:

```
1 0x6d7950617373776f726400000000000000000000000000000000000000000014
```

This value represents the raw bytes stored at slot 1. It can be decoded using Foundry's `cast parse-bytes32-string` command.

```
1 cast parse-bytes32-string 0
  x6d7950617373776f726400000000000000000000000000000000000000000014
```

Output:

```
1 myPassword
```

Recommended Mitigation:

The current architecture should be reconsidered, as secrets cannot be securely stored on-chain.

One possible approach is to encrypt the password off-chain and store only the encrypted value on-chain. However, this would require users to securely manage the decryption key off-chain. Additionally, the `getPassword` function should likely be removed, as exposing encrypted data through a retrieval function provides limited value and may encourage unsafe usage patterns.

[H-2] Missing Access Control in `PasswordStore::setPassword` Allows Unauthorized Password Changes

Description:

The `PasswordStore::setPassword` function is declared as `external` and contains no access-control checks. However, both the contract's intended functionality and its NatSpec documentation indicate that only the owner should be able to update the password.

```
1 function setPassword(string memory newPassword) external {
2     // @Audit - Missing access control
3     s_password = newPassword;
4     emit SetNewPassword();
5 }
```

Impact:

Any user can modify the stored password, breaking the contract's intended access restrictions and functionality.

Proof of Concept (PoC):

Add the following test to `PasswordStore.t.sol`:

Code

```
1 function test_anyone_can_set_password(address randomAddress) public {
2     vm.assume(randomAddress != owner);
3
4     vm.startPrank(randomAddress);
5
6     string memory expectedPassword = "myNewPassword";
7     passwordStore.setPassword(expectedPassword);
8
9     vm.startPrank(owner);
10
11     string memory actualPassword = passwordStore.getPassword();
12     assertEq(actualPassword, expectedPassword);
13 }
```

The test demonstrates that a non-owner can successfully update the password.

Recommended Mitigation:

Restrict access to the contract owner.

```
1 if (msg.sender != s_owner) {
2     revert PasswordStore__NotOwner();
3 }
```

Alternatively, consider using OpenZeppelin's `Ownable` implementation and the `onlyOwner` modifier.

Informational**[I-1] Incorrect NatSpec Documentation for `getPassword`****Description:**

The NatSpec documentation for `getPassword` includes a parameter description for a parameter that does not exist.

```
1  /*
2   * @notice This allows only the owner to retrieve the password.
3   * @param newPassword The new password to set.
4   */
5  function getPassword() external view returns (string memory) {
```

The function signature is `getPassword()` and does not accept any parameters. As a result, the NatSpec documentation is inaccurate.

Impact:

Incorrect documentation may confuse developers, auditors, integrators, and users interacting with the codebase.

Recommended Mitigation:

Remove the incorrect NatSpec parameter entry.

```
1  - * @param newPassword The new password to set.
```